PROJECT NEXUS: H.C.F.

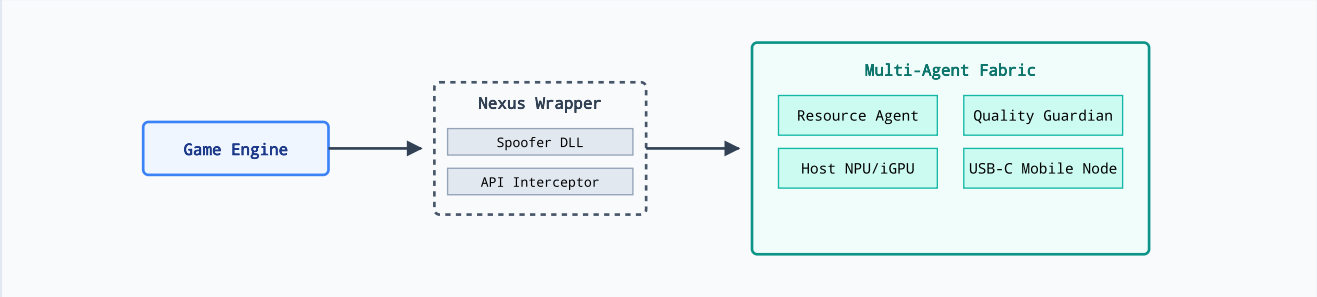
DOCUMENT ID: ENG-SPEC-01 | ARCHITECTURE: HETEROGENEOUS COMPUTE FABRIC & VIRTUALIZATION

1.0 Executive Summary

This document outlines the engineering specifications for an advanced translation and virtualization layer designed to execute demanding interactive 3D applications (AAA titles) on sub-minimum hardware specifications. Moving beyond traditional "game boosters," this system introduces a **Heterogeneous Compute Fabric (HCF)**. It operates by intercepting graphics APIs to spoof hardware capabilities, virtually bridging memory with external edge devices (mobile phones), and aggressively degrading the rendering pipeline below official minimum thresholds—all managed asynchronously by an autonomous Multi-Agent architecture.

2.0 Core Architecture Overview

The system intercepts the application at the initialization phase before memory allocation and hardware checks occur. It deplows a localized mesh network of compute nodes utilizing all available, otherwise dormant hardware (iGPU, NPU, and connected Mobile SoCs) to offload asynchronous computational tasks.



3.0 Hardware Bypass & VRAM Virtualization (The Spoofer)

Modern engines employ hard-coded telemetry during the pre-launch phase. The Spoofer module is a custom dynamic-link library (acting as a proxy for `dxgi.dll` or `d3d12.dll`) injected into the process space.

- **DXGI Adapter Forgery:** Intercepts `IDXGIFactory::EnumAdapters` and `GetDesc`. If physical VRAM is 2GB, the hook alters `DedicatedVideoMemory` to return 8,589,934,592 bytes (8GB), bypassing hard stops.
- **Feature Level spoofing:** Hooking `CheckFeatureSupport` to return `TRUE` for missing hardware capabilities (e.g., Mesh Shaders). The missing hardware pipeline is then translated via software emulation in the background.
- **Instruction Bypass:** In-memory patching (byte substitution) of CPUID checks (e.g., replacing `JNE` with `JMP` at the AVX2 verification offset) to prevent illegal instruction exceptions on older CPUs.

4.0 Extended Fabric: Mobile Edge Integration

To overcome fundamental hardware deficiencies without cloud dependency, the system utilizes an attached mobile device (via USB 3.2/4) as a low-latency Edge Compute Node. This is achieved through Memory Tiering and Split-Pipeline Execution.

Technique	Host PC Responsibility	Mobile Node Responsibility	Target Benefit
Virtual Unified Memory	Pagefault interception, streaming compressed texture assets to USB.	Caching assets in LPDDR5 RAM; returning on demand via PCIe/USB bridge.	Eliminates stuttering on systems with < 8GB system RAM.
Split-Pipeline Rendering	Geometry processing, basic rasterization, physics calculation.	AI Upscaling (NPU), Frame Generation, Anti-Aliasing interpolation.	Offloads up to 40% of GPU compute time to the mobile device.
AI Fallback Interpolation	Rendering raw frames at 10-15 FPS.	Asynchronous spatial-temporal interpolation outputting 30 FPS.	Playable framerates during extreme thermal/compute bottlenecks.

5.0 Sub-Minimum Rendering Pipeline (SMRP)

Game developers lock graphical settings to maintain visual identity. The SMRP forcefully degrades visual fidelity below these UI limits to prioritize framerate (FPS) through deep memory modification and API interception.

- **LOD Bias Forcing:** By injecting a positive Level-Of-Detail (LOD) Bias (+3.0 to +5.0) via Direct3D samplers, the engine is forced to utilize lowest-quality Mipmap textures for immediately adjacent objects. This drastically reduces VRAM bandwidth consumption while bypassing the game's official "Low" texture resolution limits.
- **Shader Nullification:** The wrapper intercepts `CreatePixelShader` API calls. When specific heavy shaders are identified via their bytecode signature (e.g., Volumetric fog, Ray-Marched reflections), the system injects an empty dummy shader that simply outputs a transparent or solid black vector. This annihilates the performance cost of effects that cannot be disabled in menus.
- **Geometry Decimation (Culling):** The system overrides draw calls (`DrawIndexedInstanced`) by analyzing bounding boxes. Any object occupying less than a defined pixel threshold on screen (e.g., distant grass, debris) is dropped from the render queue, significantly reducing CPU bottlenecks.

> **Architecture Note: Dynamic Quality Guardian**
>
> Applying SMRP globally degrades the experience unnecessarily. The **Quality Guardian Agent** monitors frametimes via Event Tracing for Windows (ETW). If frametime drops below 33.3ms (30 FPS target), it dynamically injects Shader Nullifiers and alters LOD bias in real-time. Once the computationally heavy scene concludes, it restores normal "Low" settings transparently, maintaining visual integrity where possible.

6.0 Deep Config Injection & OS-Level Scheduling

Beyond API interception, the system modifies engine initialization files directly, bypassing user interfaces. For Unreal Engine titles, the Configuration Agent parses and overwrites `Engine.ini` in real-time before thread execution, forcing hard-coded rendering variables (`r.ShadowQuality=0` , `r.ScreenPercentage=30`).

To maximize CPU utility, the Runtime Scheduler Agent modifies Windows thread affinities. It forcibly isolates the game's main render thread to the highest-performing CPU cores, while banishing OS background tasks and audio processing to efficiency cores. On systems with mixed-architecture processors or available iGPUs, asynchronous compute tasks (like physics particle updates) are forcibly routed to the iGPU via modified dispatch calls.

7.0 MVP Development Roadmap

To validate the core hypothesis and minimize initial engineering risk, development is phased. The Minimum Viable Product (MVP) focuses purely on local hardware spoofing and degradation, omitting the mobile integration for later phases.

Phase	Milestone	Core Deliverables	Risk Factor
Phase 1 (MVP)	The Spoofer & Local Wrapper	C++ DLL (DXGI Wrapper), VRAM capacity spoofing, static LOD bias forcing, single-button injection UI.	Low. API Hooking is well documented.
Phase 2	Dynamic Degradation (SMRP)	Shader Nullification via byte-code signature matching, Real-time Quality Guardian Agent.	Medium. Requires per-engine shader analysis.
Phase 3	Heterogeneous Scheduling	Routing compute tasks to iGPU + NPU. Advanced thread isolation.	High. Driver instability risks.
Phase 4	Mobile Edge Integration	USB 3.2 protocol bridge, offloading Frame Generation and Textures to phone SoC.	Extreme. Strict 10ms latency budget over USB.

8.0 Conclusion & Anti-Cheat Considerations

This architecture redefines localized compute by breaking the physical boundaries of a PC's chassis. By combining brutal API-level graphical degradation with the auxiliary compute power of mobile NPUs, previously unplayable applications become viable. **Security Note:** Injection into memory spaces monitored by Kernel-level anti-cheats (e.g., Easy Anti-Cheat, Vanguard) will result in account bans. The system must whitelist single-player or unprotected executables natively, or restrict operations purely to config injection and external driver-level upscaling for protected multiplayer titles.

9.0 Advanced Operational Resilience & Stealth

To transition this architecture from a theoretical model to a production-ready application operating in highly volatile environments, three critical operational layers must be implemented to ensure stability and evade detection.

9.1 Ring-0 Evasion Layer (Kernel Stealth)

Standard user-mode API hooking and memory patching are immediately flagged by modern intrusive Anti-Cheat mechanisms (e.g., Vanguard, BattlEye). To maintain operational security, the Injection and Spoofer agents must be elevated to operate via a custom Windows Kernel Driver (Ring 0) or utilize Hypervisor-level execution, effectively shielding memory modifications from user-level scans.

9.2 Failsafe & Hot-Swapping Mechanics

The reliance on a physical USB connection to a mobile node introduces a severe point of failure. The Runtime Scheduler features a sub-millisecond Hot-Swap protocol. Upon detecting a USB-C disconnect or mobile thermal throttling, the system instantly abandons the mobile NPU pipeline, reroutes all rendering back to the local iGPU, and forcefully applies extreme SMRP values (maximum LOD bias) to prevent the application from crashing while awaiting reconnection.

9.3 Ultra-Low Latency Transport Protocol

Transporting uncompressed graphical data to the phone exceeds the critical 16.6ms frametime budget. The system bypasses standard USB data transfers by interfacing directly with hardware video encoders (e.g., NVENC, Intel QuickSync). It compresses the required data into a high-speed, low-latency video stream that, coupled with isolated motion vectors, is transmitted and processed via a custom protocol (similar to Parsec), ensuring a complete round-trip latency of under 10ms.